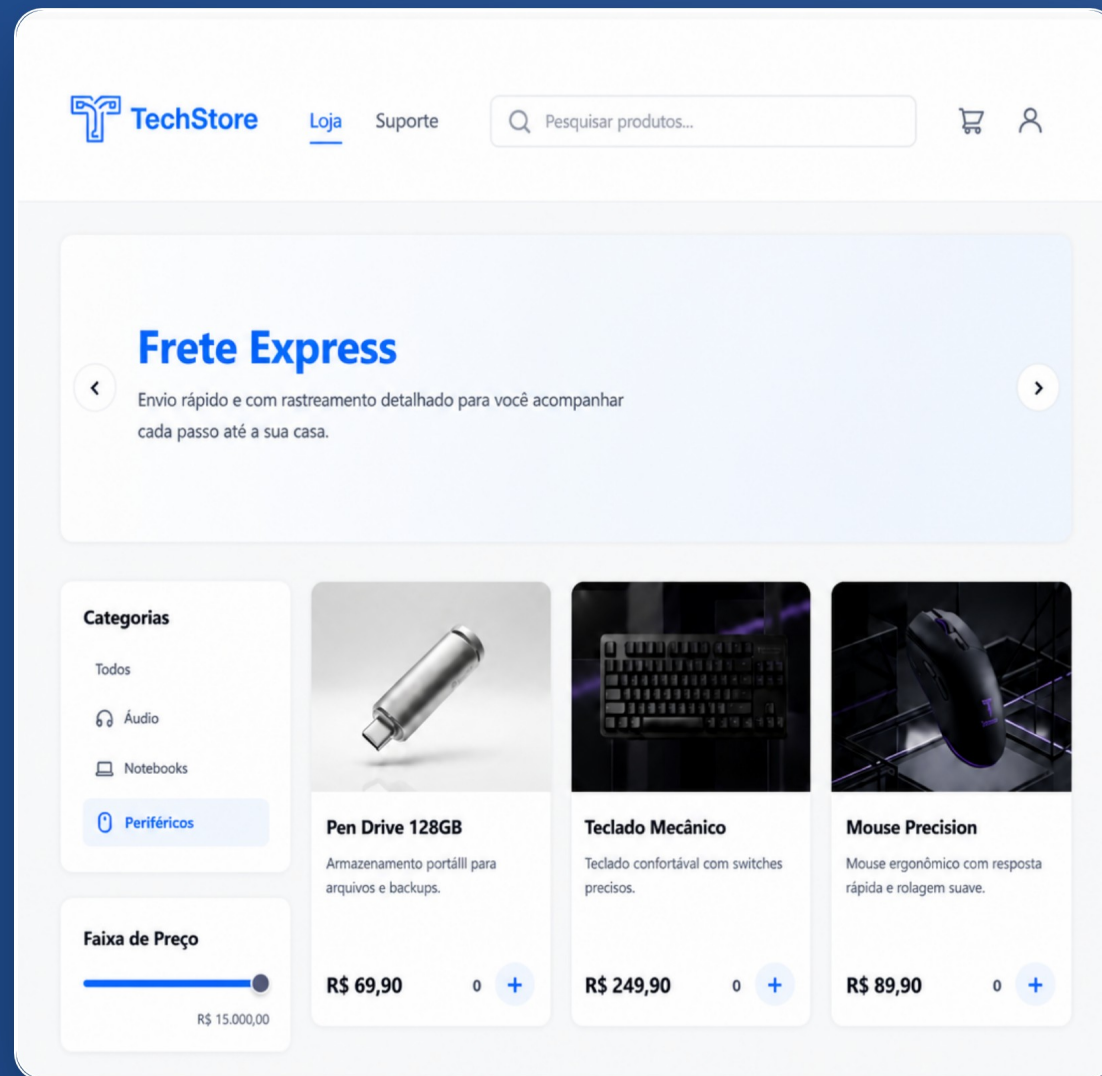


TechStore

Persistência e Gestão de Dados

"Uma loja demo onde os dados sobrevivem ao ciclo de vida efêmero da infraestrutura."



O problema e quem é impactado

Banco de dados acoplado ao contêiner da aplicação: cada reinício vira perda de dados.

SEM PERSISTÊNCIA · CENÁRIO DE RISCO

Acoplamento efêmero

O banco sobe e desce junto com o contêiner da aplicação.

Perda total de dados

Reiniciar, atualizar ou recriar o contêiner apaga tabelas, sessões e produtos.

Impacto operacional

Inviabiliza a operação, com perda irreparável a cada manutenção.

QUEM É IMPACTADO

C

Cliente

Perde o carrinho e a sessão a cada reinício, e repete o cadastro.

A

Administração

Perde o histórico de pedidos e não consegue auditar vendas.

O

Operação

Cada manutenção vira risco de negócio, sem garantia de retorno.

A solução proposta

O dado tem ciclo de vida próprio, independente do contêiner da aplicação.

CAMADA DE DADOS ISOLADA

- Volume nomeado `pg_data` montado fora do contêiner, com **ciclo de vida próprio**. É ele que sobrevive à recriação da stack.
- **Banco de dados** em contêiner dedicado, independente da aplicação.
- Rede dedicada `techstore_net`, com a porta do banco restrita ao **localhost**.

INTEGRIDADE DOS DADOS

- **5 constraints** `CHECK` e 4 índices no PostgreSQL, declarados em migração versionada.
- **Preço e nome** do produto gravados no pedido no momento da compra, como snapshot em `pedido_itens`.
- A busca por nome **não percorre o banco**: é filtro no catálogo já carregado, no frontend.

Como o MVP funciona

Um comando sobe a stack inteira. O volume sobrevive ao ciclo de vida dos contêineres.

Comando único

`docker compose up -d --build`

Ordem de subida

Banco, depois backend, depois frontend. Cada serviço espera o anterior.

Bootstrap

`entrypoint.sh` aplica migrações e popula o banco na subida.

Healthcheck

A API só recebe tráfego depois que o banco responde *healthy*.

Acesso

`localhost:3001` interface · `localhost:3002` API

Seed

Idempotente: 8 produtos e 2 usuários, sem duplicar registros na reexecução.

CICLO DE VIDA INDEPENDENTE

`docker compose down` volume preservado

`docker compose up -d` volume reutilizado

`docker compose down -v` volume removido, dados apagados

O volume sobrevive ao `down`, e o banco é recriado a partir dele no `up` seguinte.

O dado sobrevive à recriação

The screenshot displays the TechStore application interface. The main header includes the TechStore logo, navigation links for 'Loja' and 'Suporte', a shopping cart icon, and a user greeting 'Olá, Gerente TechStore'. The left sidebar contains a 'TechStore' section with a 'GESTÃO TECHSTORE' menu, including 'Dashboard', 'Estoque' (highlighted), 'Pedidos', 'Clientes', and 'Pix'. The central area is titled 'Gerenciamento de Estoque' with a subtitle 'Monitore e gerencie os produtos cadastrados.' and a search bar 'Buscar por nome...'. Below this is a table of products with columns 'PRODUTO', 'ESTOQUE', 'STATUS', and 'AÇÕES'. The table lists five products: 'Toca-discos Vintage' (SKU: SKU-0008), 'Pen Drive 128GB' (SKU: SKU-0007), 'Teclado Mecânico' (SKU: SKU-0006), 'Mouse Precision' (SKU: SKU-0005), and 'Note Slim' (SKU: SKU-0004). All products are currently 'Em Estoque'. A modal window titled 'Novo Produto' is open in the center, containing fields for 'Nome do produto', 'SKU' (with example 'SBX-2026'), 'Categoria' (set to 'Notebooks'), 'Preço (R\$)', 'Estoque', 'URL da imagem' (with example '/img/produto.jpg'), and 'Descrição'. The modal has 'Cancelar' and 'Cadastrar' buttons. A '+ Novo Produto' button is also visible in the top right of the main content area.

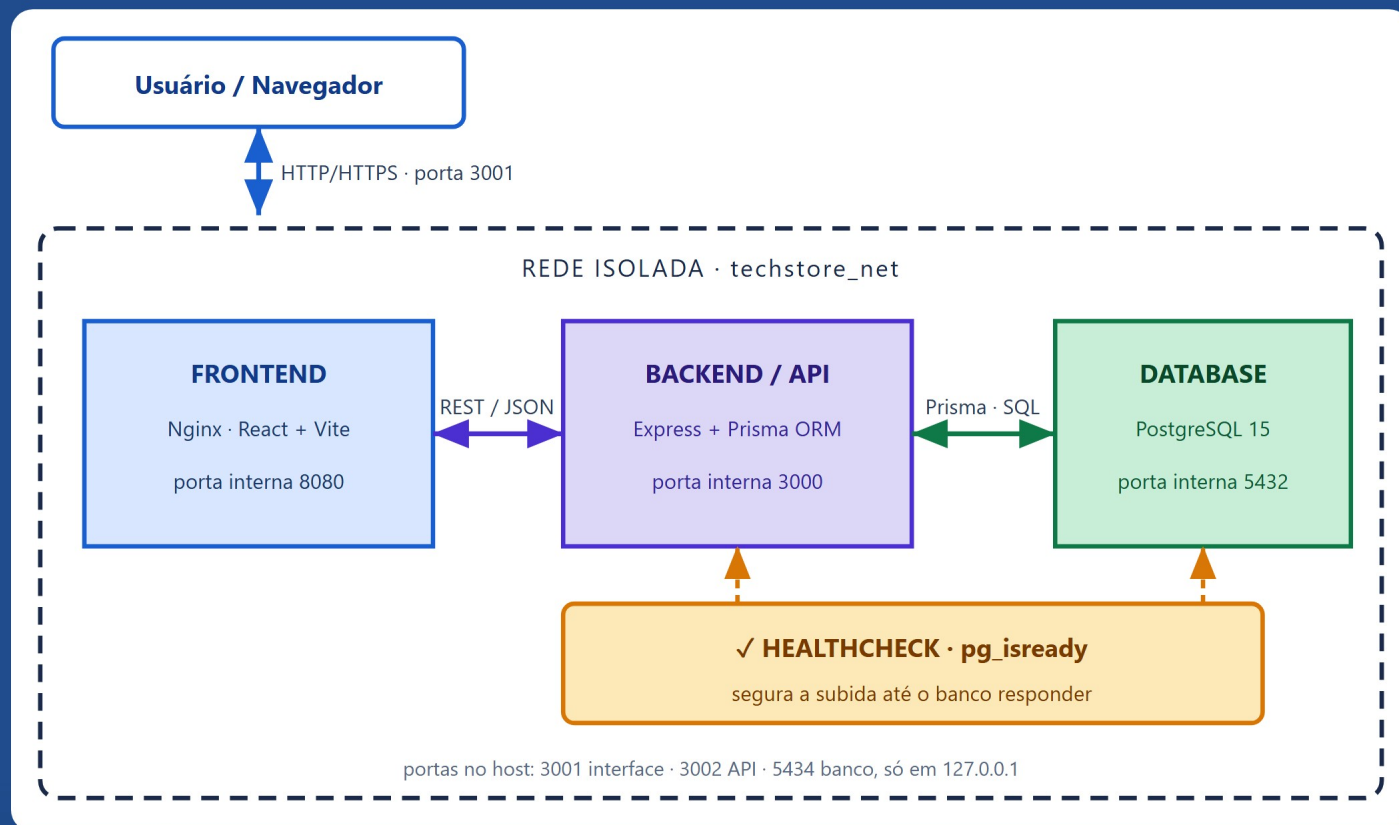
PRODUTO	ESTOQUE	STATUS	AÇÕES
 Toca-discos Vintage SKU: SKU-0008	10 un.	● Em Estoque	Editar Excluir
 Pen Drive 128GB SKU: SKU-0007	80 un.	● Em Estoque	Editar Excluir
 Teclado Mecânico SKU: SKU-0006	35 un.	● Em Estoque	Editar Excluir
 Mouse Precision SKU: SKU-0005	60 un.	● Em Estoque	Editar Excluir
 Note Slim SKU: SKU-0004	18 un.	● Em Estoque	Editar Excluir

Saída Console de Depuração Terminal Portas

PS C:\Users\L\techstore-db-mvp> docker compose down

pwsh ⚠ + ▾ 🗑 ...

Como foi construído, com o quê e por quê



PRÁTICAS E FERRAMENTAS DEVOPS

- **Docker e Docker Compose**
stack como código
- **Healthcheck e readiness**
sem corrida na subida
- **Bootstrap automatizado**
seed idempotente
- **Volume nomeado**
driver local, declarado no compose
- **CI/CD · 3 workflows**
`ci.yml` `test-docker-stack.yml` `protecao-main.yml`
- **Gestão de segredos**
JWT gerado no primeiro boot, com permissão 600

DECISÕES TÉCNICAS E POR QUÊ

- **PostgreSQL 15**
transação e CHECK no banco, não no código
- **Express com Prisma**
o schema gera a migração versionada
- **Nginx como proxy**
a API não precisa ficar exposta
- **Rede e portas restritas**
o banco só responde dentro da rede
- **Checkout transacional**
sem janela entre ler e escrever

Resultados obtidos e próximos passos

RESULTADOS COM O MVP

- 01 Arranque zero-config**
Stack inteira em um comando Docker Compose.
- 02 Bootstrap idempotente**
8 produtos e 2 usuários sem duplicar registros.
- 03 Persistência e integridade verificadas**
Dados sobrevivem à recriação da stack; o checkout impede oversell.
- 04 Qualidade comprovada**
10 suítes e 35 testes no backend, 6 tabelas e 16 endpoints REST.
- 05 Backup e restauração verificados**
Scripts que restauram num banco separado e comparam as contagens.

PRÓXIMOS PASSOS E EVOLUÇÃO

- 01** Reserva e expiração de estoque
- 02** Histórico de pedidos e pagamentos
- 03** Ledger de estoque para auditoria
- 04** Migrações sem indisponibilidade
- 05** Banco não relacional complementar, para cache e auditoria
- 06** Observabilidade: logs estruturados e métricas de latência

OBRIGADO PELA ATENÇÃO

Obrigado!

A perda de dados é um problema real de infraestrutura, e a resposta foi uma decisão de arquitetura.

LG

Lays Gomes

Apresentação

FA

Fellype Augusto

Apresentação

IT

Ivan Teotônio

Apresentação

VK

Victor Kazu

Apresentação

TechStore · MVP de Persistência e Gestão de Dados · **DevOps** · **Aponti** · 05/10/26

github.com/laas26/techstore-db-mvp